LANGGRAPH

Sequential Workflows

Video 5 -- Comprehensive Visual Notes

Detail	Value
Topic	Sequential Workflows -- Linear Graph Pipelines
Pattern	Prompt Chaining (No branching, no loops)
Prerequisites	Video 4: State, Nodes, Edges, Reducers
Key Library	pip install langgraph langchain-openai
Source	CampusX -- Agentic AI using LangGraph

>> What You Will Learn

The 4-step recipe for building ANY LangGraph workflow. How State flows and evolves through a sequential pipeline. A hands-on Blog Generator case study. Extra code templates, comparisons, and revision aids not in the original video.

1. Core Concept: What is a Sequential Workflow?

A sequential workflow is a **linear pipeline** where tasks execute one after another in a fixed order. There is **no branching** (if/else), **no parallelism**, and **no looping**. It is the simplest and most predictable graph topology in LangGraph.

Sequential Workflow -- The Simplest Graph Pattern

```
[ START ] =====> [ Node A ] =====> [ Node B ] =====> [  END  ]
  Fixed order  |  No branches  |  No loops  |  Deterministic
```

Analogy: Think of a sequential workflow like an **assembly line** in a factory. Each station (node) performs one task on the product (state), then passes it forward. The product never goes backward and never skips a station.

+ Extra: When to Use Sequential Workflows

Use them when your task naturally decomposes into ordered steps with no decision points. Examples: data pipelines (extract -> transform -> load), content generation (outline -> draft -> polish), form processing (validate -> enrich -> store). If you ever need if/else or retry logic, use Routing or Evaluator-Optimizer patterns instead.

2. The 4-Step Recipe for Building a Graph

Every LangGraph workflow, regardless of complexity, follows these four architectural steps. Master this recipe and you can build any graph.

1

Define the State (The "Memory")

The State is a shared, mutable dictionary (TypedDict) that persists across the entire workflow. As execution progresses node to node, this state accumulates data. Every node can read and write to it.

```
from typing import TypedDict

class BlogState(TypedDict):
    topic: str         # User input
    outline: str       # Generated by Node 1
    final_post: str    # Generated by Node 2
```

! Extra: Why TypedDict and Not a Regular Dict?

TypedDict provides: (1) Type safety -- IDE catches typos like state['tpic'] instantly. (2) Autocompletion -- your editor knows exactly which keys exist. (3) Schema enforcement -- LangGraph validates the state shape at compile time. A regular dict gives you none of these protections.

2 Define the Nodes (The "Workers")

A Node is a unit of work -- a standard Python function. It follows a strict contract: receives the current State, performs a task, and returns a PARTIAL update. LangGraph merges it automatically.

```
def generate_outline(state: BlogState) -> dict:
    # Receives full state, returns only what changed
    topic = state["topic"]
    outline = llm.invoke(f"Write an outline for: {topic}")
    return {"outline": outline.content}  # Partial update!

def generate_blog(state: BlogState) -> dict:
    topic = state["topic"]
    outline = state["outline"]  # Read from accumulated state
    blog = llm.invoke(f"Write blog about {topic} using: {outline}")
    return {"final_post": blog.content}
```

! Key Rule: Partial Updates

Never return the ENTIRE state from a node. Only return the keys that changed. LangGraph performs a shallow merge: it takes the existing state dict and updates only the keys you returned. This is memory-efficient and prevents accidentally overwriting data.

Node Function Contract

Aspect	Requirement	Example
Input	Receives the full State dict	state: BlogState
Logic	Any Python code (LLM, API, math)	llm.invoke(prompt)
Output	Returns a PARTIAL dict update	{"outline": "..."}
Side Effects	Allowed but not recommended	Logging, DB writes

3 Define the Edges (The "Connections")

Edges dictate control flow -- which node runs next. In a sequential workflow, edges are always static. LangGraph provides two sentinels: START (entry) and END (exit).

```
from langgraph.graph import StateGraph, START, END

graph = StateGraph(BlogState)

# Add nodes
graph.add_node("generate_outline", generate_outline)
graph.add_node("generate_blog", generate_blog)

# Wire edges (linear flow)
graph.add_edge(START, "generate_outline")
graph.add_edge("generate_outline", "generate_blog")
graph.add_edge("generate_blog", END)
```

Edge Types Comparison

Edge Type	Behavior	Used In	This Video?
Normal Edge	Always goes A -> B	Sequential workflows	Yes
Conditional Edge	if/else branching	Routing, Eval-Optimizer	No
Cyclic Edge	Loops back to a prior node	Evaluator-Optimizer	No

4

Compile and Run

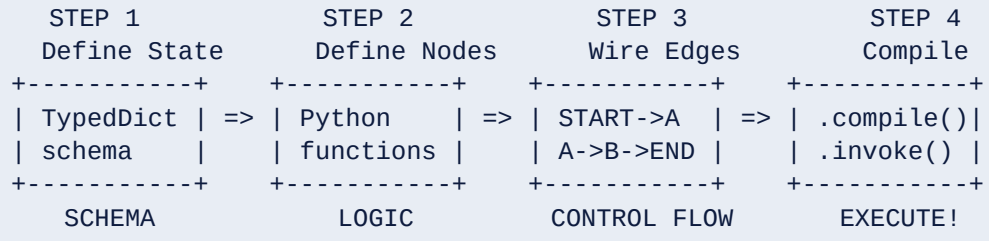
Before running, compile the graph. This freezes the structure, validates for errors (orphaned nodes, missing edges), and creates a Runnable object you can invoke.

```
# Compile
app = graph.compile()

# Run with initial state
result = app.invoke({"topic": "AI in Healthcare",
                     "outline": "", "final_post": ""})

# Access ALL intermediate + final results
print(result["topic"])       # Original input preserved
print(result["outline"])     # Intermediate output preserved!
print(result["final_post"])  # Final output
```

? What Does Compilation Check?

Orphaned nodes -- nodes added but never connected via edges. Missing START edge -- graph must know where to begin. Unreachable END -- at least one path must reach END. Type mismatches -- validates State schema consistency.

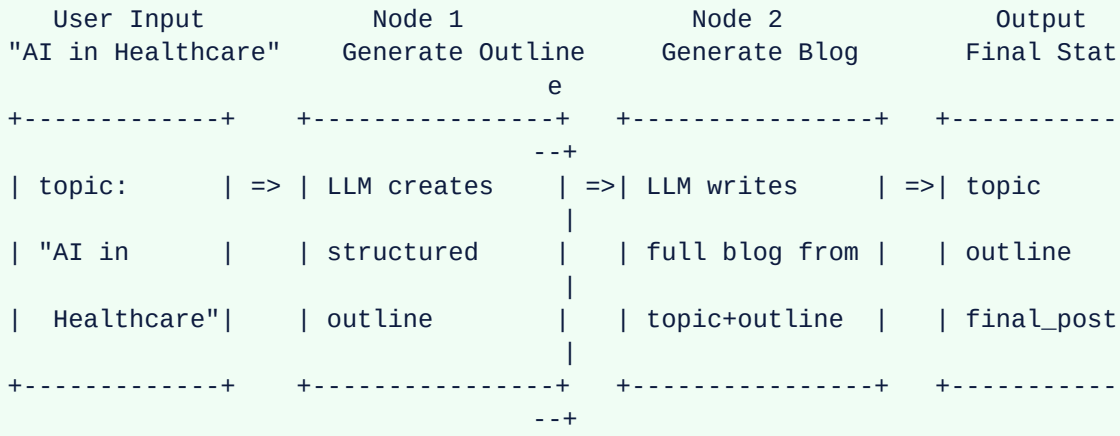
Complete 4-Step Recipe -- From Code to Execution



3. Case Study: The Blog Generator (Prompt Chaining)

The lecture demonstrates sequential logic using a **Prompt Chaining** pattern. Two LLM calls are chained: one generates an outline, the next uses that outline to write a full blog post.

Blog Generator -- Prompt Chaining Flow



State Evolution Through the Pipeline

Watch How State Grows at Each Step

```
                INITIAL STATE (user input):
{ topic: "AI in Healthcare", outline: "", final_post: "" }
                                        |
                                        V
             AFTER NODE 1 (generate_outline):
               { topic: "AI in Healthcare",
    outline: "1. Intro  2. Use Cases  3. Challenges...",
                      final_post: "" }
                                        |
                                        V
              AFTER NODE 2 (generate_blog):
               { topic: "AI in Healthcare",
    outline: "1. Intro  2. Use Cases  3. Challenges...",
     final_post: "AI is transforming healthcare by..." }
==> ALL intermediate data is preserved in the final output!
```

Key Advantage Over LangChain

Aspect	LangChain (Simple Chain)	LangGraph (Sequential)
Intermediate Data	Lost after chain runs	Preserved in State forever
Debugging	Hard -- can't see middle steps	Easy -- inspect any key anytime
State Access	Only final output available	Full state: topic+outline+blog
Extensibility	Must rewrite chain to add steps	Just add a node + edge
Memory / History	Manual setup needed	Built into the framework

! Why This Matters for Real Projects

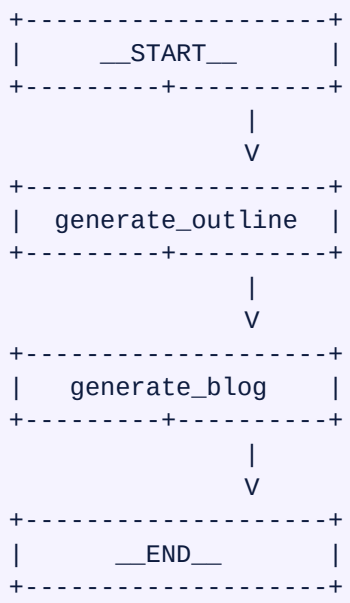
In production, you often need to: (1) Log intermediate outputs for auditing, (2) Display a progress indicator showing each step, (3) Retry from a specific step if something fails, (4) Let users edit intermediate results before continuing. LangGraph's persistent State makes ALL of these trivial.

4. Graph Visualization

LangGraph provides built-in tools to render your graph as a visual diagram. This confirms edges are wired correctly before you run anything.

```
# Visualize in Jupyter Notebook
from IPython.display import Image, display
display(Image(app.get_graph().draw_mermaid_png()))

# Or save to file
png_bytes = app.get_graph().draw_mermaid_png()
with open("graph.png", "wb") as f:
    f.write(png_bytes)
```

What the Visualization Looks Like



5. Extra: Complete Working Code Template

A **complete, runnable** Blog Generator. Copy-paste to get started immediately.

```
# === COMPLETE BLOG GENERATOR WITH LANGGRAPH ===
# pip install langgraph langchain-openai

from typing import TypedDict
from langgraph.graph import StateGraph, START, END
from langchain_openai import ChatOpenAI

# --- STEP 1: Define State ---
class BlogState(TypedDict):
    topic: str
    outline: str
    final_post: str

# --- Initialize LLM ---
llm = ChatOpenAI(model="gpt-4o-mini", temperature=0.7)

# --- STEP 2: Define Nodes ---
def generate_outline(state: BlogState) -> dict:
    prompt = f"Create outline for blog: {state['topic']}"
    response = llm.invoke(prompt)
    return {"outline": response.content}

def generate_blog(state: BlogState) -> dict:
    prompt = (f"Write blog about {state['topic']} "
              f"following outline:\n{state['outline']}")
    response = llm.invoke(prompt)
    return {"final_post": response.content}

# --- STEP 3: Build Graph + Edges ---
graph = StateGraph(BlogState)
graph.add_node("generate_outline", generate_outline)
graph.add_node("generate_blog", generate_blog)
graph.add_edge(START, "generate_outline")
graph.add_edge("generate_outline", "generate_blog")
graph.add_edge("generate_blog", END)

# --- STEP 4: Compile & Run ---
app = graph.compile()
result = app.invoke({"topic": "AI in Healthcare",
                     "outline": "", "final_post": ""})
print(result["outline"])
print(result["final_post"])
```

6. Extra: Extending the Pipeline (3-Node)

Need a new step? Just add a node and an edge. Here is how to add a "Polish" step:

Extended 3-Node Pipeline

```
[START] => [generate_outline] => [generate_blog] => [polish_post] => [END
                                                                      ]
                                                                (NEW!)
```

```python
# Add a new state key
class BlogStateV2(TypedDict):
    topic: str
    outline: str
    draft: str          # Renamed from final_post
    final_post: str     # Now the POLISHED version

# Add the new node
def polish_post(state: BlogStateV2) -> dict:
    prompt = (f"Improve this blog post. Fix grammar, "
              f"add transitions:\n{state['draft']}")
    response = llm.invoke(prompt)
    return {"final_post": response.content}

# Wire it in
graph.add_edge("generate_blog", "polish_post")
graph.add_edge("polish_post", END)
```

7. Extra: Common Pitfalls and Best Practices

Pitfall	What Goes Wrong	Fix
Returning full state	Overwrites keys with stale data	Only return changed keys
Forgetting START edge	No entry point; compile error	Always add_edge(START, first)
Orphaned nodes	Node defined but not connected	Compile step catches this
Empty initial state	Nodes crash on missing keys	Pass all keys with defaults
No error handling	One failure kills pipeline	try/except in node logic
Huge state objects	Memory grows each node	Summarize large outputs

>> Pro Tip: Debug with State Inspection

Add a debug node: def debug(state): print(state); return {}. Insert between any two nodes during development. Remove before production. This is LangGraph "print debugging" and works because State is always available.

8. Where Sequential Workflows Fit

Pattern (Video 4)	Loops?	Branching?	Sequential?	Complexity
Sequential (THIS VIDEO)	No	No	Yes	* Easy
Routing	No	Yes	No	** Medium
Parallelization	No	Fan-out	No	*** Hard
Orchestrator-Worker	No	Dynamic	No	**** V.Hard
Evaluator-Optimizer	Yes	Yes	No	***** Expert

>> The Learning Path

Video 4 taught concepts (State, Nodes, Edges). This Video 5 is your first hands-on implementation using the simplest pattern. Upcoming videos add conditional edges (routing), parallel execution, and the full agentic loop. The 4-step recipe stays the same -- only the edge topology changes.

9. Quick Revision Checklist

Test yourself. Can you answer each from memory?

Tag	Question
Define	What is a sequential workflow and when should you use one?
State	Why TypedDict instead of a regular dict?
Nodes	What is the input/output contract for every node?
Partial	Why return PARTIAL dicts instead of full state?
Edges	What are START/END and how do you wire linear flow?
Compile	What errors does compilation catch?
Advantage	How does LangGraph preserve intermediate outputs?
Extend	How would you add a 3rd node to the Blog Generator?

Cheat Sheet: Core Code Patterns

What	Code
Define State	class MyState(TypedDict): ...
Define Node	def my_node(state: MyState) -> dict: ...
Create Graph	graph = StateGraph(MyState)
Add Node	graph.add_node("name", function)
Add Edge	graph.add_edge("from", "to")
Start Edge	graph.add_edge(START, "first_node")
End Edge	graph.add_edge("last_node", END)
Compile	app = graph.compile()
Run	result = app.invoke({"key": "value"})
Visualize	app.get_graph().draw_mermaid_png()

What's Next?

In the next video, you will learn **Conditional Edges** and **Routing** -- adding if/else logic to your graphs. The 4-step recipe stays the same; only Step 3 gets more powerful. Make sure you can write the Blog Generator from memory before moving on!